Assignment

Bachelor of Computer Applications

Semester III

Database Management System

Topic: DATABASE DESIGN ANALYSIS

BY

Name: M Blessy

Registration Number: 2511021240067

DEPARTMENT OF COMPUTER APPLICATIONS

Alliance university

Chikkahagade cross, Anekal, Bangaluru 562106

DATABASE DESIGN ANALYSIS

Hospital Management System

1. Introduction

The Hospital Management System is designed to manage hospital information, including departments, doctors, patients, appointments, rooms, admissions, prescriptions, medicines, laboratory tests, and bills.

The database is created using MySQL. It contains 11 tables: Department, Doctor, Patient, Appointment, Room, Admission, Prescription, Medicine, Prescription_Detail, Lab_Test, and Bill.

Primary keys are used to uniquely identify records, while foreign keys establish relationships between related tables. The database also uses constraints such as UNIQUE, CHECK, and DEFAULT to maintain data accuracy and consistency.

The purpose of this analysis is to identify data redundancy, examine functional dependencies, explain insertion, deletion, and update anomalies, and suggest improvements to the database design.

2. Analysis of Data Redundancy

Data redundancy means storing the same information unnecessarily in multiple places. It can increase storage requirements and cause inconsistencies when data is inserted, updated, or deleted.

The Hospital Management System reduces redundancy by storing different types of information in separate tables.

Examples from the database include:

- **Doctor information:** Doctor details are stored in the Doctor table. Other tables refer to doctors using doctor_id.
- **Patient information:** Patient details are stored in the Patient table. Appointments, admissions, prescriptions, and bills refer to patients using patient_id.
- **Medicine information:** Medicine names, manufacturers, prices, and stock quantities are stored in the Medicine table. The Prescription_Detail table refers to medicines using medicine_id.
- **Room information:** Room numbers, types, floors, and daily charges are stored in the Room table. The Admission table refers to rooms using room_id.
- **Department information:** Department names and contact information are stored in the Department table, while doctors refer to departments using department_id.

For example, the doctor's name and phone number do not need to be repeated in every appointment record. The Appointment table stores the doctor_id, and the doctor's details can be retrieved using a JOIN operation.

However, some potential redundancy remains. The Prescription table stores appointment_id, patient_id, and doctor_id together. If the patient and doctor are already determined by the appointment, storing all three values may duplicate information and allow inconsistent combinations.

Similarly, the Bill table stores patient_id along with appointment_id and admission_id. These relationships should be checked to ensure that the bill's patient matches the patient associated with the linked appointment or admission.

Therefore, the database reduces redundancy through separate tables, but additional integrity rules could improve consistency.

3. Functional Dependencies

A functional dependency describes a relationship in which one attribute or group of attributes determines another attribute or group of attributes.

It is represented as:

X → Y

This means that the value of X determines the value of Y.

In the Hospital Management System, the following functional dependencies can be identified from the intended meaning of the tables.

Table	Functional dependency
Department	department_id → department_name, floor_no, phone_no, head_doctor_id
Doctor	doctor_id → name, specialization, department_id, phone_no, email, experience_years, consultation_fee
Patient	patient_id → name, dob, gender, phone_no, address, blood_group, emergency_contact
Appointment	appointment_id → patient_id, doctor_id, appointment_date, appointment_time, status, reason
Room	room_id → room_number, room_type, floor_no, daily_charge, status
Admission	admission_id → patient_id, room_id, doctor_id, admission_date, discharge_date, admission_reason, status
Prescription	prescription_id → appointment_id, patient_id, doctor_id, date_issued, notes
Medicine	medicine_id → medicine_name, manufacturer, unit_price, stock_quantity
Prescription_Detail	prescription_detail_id → prescription_id, medicine_id, dosage, duration_days, quantity
Lab_Test	test_id → patient_id, doctor_id, test_name, test_date, result, status
Bill	bill_id → patient_id, appointment_id, admission_id, bill_date, total_amount, payment_status, payment_mode

The primary key of each table determines the other attributes in that table.

There are also additional dependencies implied by the design:

- department_name → department_id because department names have a UNIQUE constraint.
- doctor email → doctor_id because the doctor's email has a UNIQUE constraint.
- room_number → room_id because room numbers have a UNIQUE constraint.

These dependencies help prevent duplicate department names, doctor email addresses, and room numbers.

Important limitation: The SQL schema does not define a rule requiring Prescription.patient_id and Prescription.doctor_id to match the patient and doctor recorded in its linked appointment. Similarly, it does not enforce that a bill's patient matches the patient linked to its appointment or admission. These consistency rules should be considered during improvement.

4. Insertion Anomaly

An insertion anomaly occurs when information cannot be inserted into a table without providing unrelated information, or when the same information must be repeated unnecessarily.

In the Hospital Management System, separate tables reduce this problem. For example, a new department can be inserted into the Department table without first inserting a doctor into the Doctor table because head_doctor_id is allowed to be NULL.

However, the database could still have practical limitations. If the hospital wants to record a prescription for a patient, the corresponding patient, doctor, and appointment records must already exist because of the foreign-key relationships.

Example: A prescription cannot refer to a non-existent appointment. The appointment must be inserted first.

This is a valid referential-integrity rule, rather than necessarily a design error. A better design should distinguish between information that genuinely requires a related record and information that should be independently insertable.

5. Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes important information about another entity.

The current database uses separate tables to reduce this risk. For example, deleting an appointment does not automatically delete the patient's record because patient details are stored separately.

Foreign keys also restrict deletion when related records exist, unless an appropriate cascading action has been configured. The supplied schema does not define ON DELETE CASCADE, so MySQL normally prevents deletion of a referenced parent record.

Example: A patient who has appointments, admissions, or bills linked to their patient ID cannot simply be deleted while those references remain.

This protects related records, although the hospital should establish a suitable policy for inactive patients and historical records. Keeping historical information is often preferable to deleting it.

6. Update Anomaly

An update anomaly occurs when the same information is stored in multiple places and must be changed in every location. If one copy is missed, the database becomes inconsistent.

The database reduces this problem by storing doctor, patient, medicine, department, and room details in their own tables.

For example, if a doctor's phone number changes, it can be updated once in the Doctor table instead of updating every appointment record.

However, the Prescription table stores both appointment_id and patient_id and doctor_id. If the appointment identifies one patient and doctor but the prescription contains different IDs, the database could store inconsistent information because the schema does not enforce their agreement.

Therefore, additional integrity rules or a revised prescription design can reduce this risk.

7. How Poor Database Design Causes Anomalies

Poor database design can cause the following problems:

1. **Data duplication:** Repeating patient or doctor details in multiple tables increases redundancy.
2. **Insertion problems:** New information may require unrelated records to be inserted first.
3. **Deletion problems:** Removing a record may accidentally remove information that should be retained.
4. **Update problems:** Repeated information may be updated in one place but not in another.
5. **Inconsistent relationships:** A prescription or bill may refer to a patient or doctor who does not match the associated appointment.
6. **Difficult maintenance:** Repeated or inconsistent data makes it harder to manage hospital records accurately.

These problems can affect billing, patient history, appointment management, and the reliability of hospital reports.

8. Proposed Improvements

The following improvements can make the Hospital Management System more reliable.

8.1 Maintain separate tables

Continue storing patients, doctors, departments, rooms, medicines, appointments, and bills in separate tables. This structure reduces unnecessary repetition and makes updates easier.

8.2 Strengthen referential integrity

Add suitable rules or application-level validation to ensure that the patient and doctor associated with a prescription match the linked appointment. Apply similar validation to bills linked to appointments or admissions.

8.3 Use appropriate constraints

The database already uses primary keys, foreign keys, UNIQUE constraints, and CHECK constraints. These should be retained and expanded where necessary.

For example, rules could ensure that discharge dates are not earlier than admission dates and that required fields are not left NULL.

8.4 Handle historical records carefully

Instead of deleting patient, doctor, or billing history unnecessarily, use suitable record-retention or inactive-status policies. This helps preserve important hospital records.

8.5 Review normalization

The tables are structured around individual entities and their primary keys, which helps reduce redundancy. The design broadly follows the principles of normalization. However, the functional dependencies and repeated relationships should be reviewed to ensure that the design satisfies the intended integrity rules.

8.6 Improve validation of bills and prescriptions

The database should ensure that bills and prescriptions are associated with the correct patients and relevant appointments or admissions. This can be implemented through suitable database constraints, triggers, or application-level validation, depending on the final design.

9. Conclusion

The Hospital Management System uses a relational database design with 11 tables. Primary keys identify individual records, foreign keys connect related tables, and UNIQUE and CHECK constraints help maintain data quality.

Separating patient, doctor, department, room, medicine, appointment, and billing information reduces unnecessary data redundancy and helps prevent insertion, deletion, and update anomalies.

Some potential inconsistencies remain because the schema does not enforce every relationship between prescriptions, appointments, patients, doctors, and bills. By

strengthening referential integrity, validating related records, and reviewing functional dependencies, the database can become more accurate, consistent, and easier to maintain.

Overall, the current design provides a useful foundation for a Hospital Management System, with further improvements possible to strengthen data integrity and reduce the risk of anomalies.